

Enunciado - Práctico Integrador 2026

Cada grupo deberá diseñar, construir y poner en funcionamiento un sistema basado en microservicios sobre un dominio nuevo. El sistema deberá estar testeado, ser observable, contemplar mecanismos de tolerancia a fallas e integrarse de manera efectiva con el sistema desarrollado por otro grupo.

El Trabajo Práctico Integrador recorre todos los contenidos abordados durante la materia. El enunciado establece **qué** condiciones deberá cumplir el sistema, pero no determina **cómo** implementarlas. Las decisiones de diseño, arquitectura y tecnología serán responsabilidad de cada grupo, que deberá documentarlas, justificarlas y defenderlas durante las distintas instancias de evaluación.

1. Dominio

Cada grupo deberá seleccionar un dominio distinto al utilizado en el Trabajo Práctico de Desarrollo de Software y distinto al elegido por los demás grupos. La propuesta deberá ser aprobada por los profesores.

El dominio elegido no será un sistema compuesto únicamente por operaciones de alta, baja, modificación y consulta. La acción principal deberá involucrar restricciones, estados, validaciones o consecuencias relevantes para el negocio.

Ejemplo:

Dominio	Entidad principal	Acción principal	Característica relevante
Turnos médicos	Profesional / turno	Reservar un turno	Dos personas no pueden tomar el mismo turno; picos de demanda

2. Requisitos de Arquitectura y Desarrollo

2.1 Arquitectura de servicios

El backend deberá estar compuesto por **al menos tres microservicios y un api gateway**.

Al menos dos servicios deberán tener patrones de arquitectura diferentes, trabajados durante la materia.

2.2 Frontend

El sistema dispondrá de una interfaz web que centralizará la interacción del usuario y dará soporte a todas las funcionalidades del proyecto.

2.3 Balanceo de Carga

El sistema deberá incorporar balanceo de carga sobre al menos un servicio desplegado en dos o más instancias. El mecanismo deberá reconocer cuándo una instancia no se encuentra disponible. La distribución y el estado de las instancias deberán poder comprobarse mediante la información de observabilidad del sistema.

2.5 Comunicación síncrona y asíncrona

El grupo deberá implementar comunicación **síncrona y asíncrona**. Para la comunicación síncrona, deberá definir timeout, manejo de errores, respuestas tardías o ausentes y política de reintentos. Para la comunicación asíncrona, deberá utilizar mensajería, incluyendo la publicación y consumo de un evento de dominio y el tratamiento de mensajes fallidos.

2.6 Búsqueda

Una entidad del dominio deberá contar con una funcionalidad de búsqueda implementada mediante un motor de búsqueda. La búsqueda deberá soportar paginación, filtros relevantes para el dominio y al menos un criterio de ordenamiento. El índice deberá mantenerse sincronizado con los cambios producidos en el sistema y el grupo deberá definir el retraso máximo tolerable entre una modificación en los datos fuente y su aparición en los resultados.

2.7 Caché

El sistema deberá incorporar caché en al menos un flujo de lectura relevante. La elección deberá basarse en una necesidad observable del sistema y no únicamente en el cumplimiento formal del requisito. Se deberá incluir evidencia por medio de métricas que permita observar el impacto de la misma.

2.8 Consistencia

Cada servicio deberá seleccionar el tipo de almacenamiento que mejor responda a sus patrones de acceso. El sistema deberá utilizar **al menos dos tipos de almacenamiento diferentes** (uno relacional, otro no relacional), volumen, necesidad de consulta y garantías de consistencia. Deberá existir **al menos una operación que requiera garantías de consistencia**, de manera que no pueda duplicarse ni perderse información.

2.9 Resiliencia

El grupo deberá identificar las dependencias de cada servicio y definir qué ocurrirá cuando una de ellas falle o responda con lentitud, para esto deberá implementarse **al menos un mecanismo de protección frente a fallas**.

El comportamiento deberá demostrarse mediante una caída provocada y quedar registrado en el [`POSTMORTEM.md`](#).

2.10 Observabilidad

El sistema deberá permitir comprender el estado de una operación sin depender de la inspección manual de cada componente. Como mínimo, deberá incluir como mínimo logs estructurados, métricas y un tablero que permita observar su estado general:

2.11 Testing

El proyecto deberá incluir, como mínimo:

1. Tests unitarios sobre reglas de negocio relevantes, superando el porcentaje que represente calidad;
2. Tests de integración por servicio contra una dependencia real, como un almacenamiento o un sistema de mensajería;
3. Tests de carga cuyos resultados queden documentados y analizados.

2.12 Puesta en marcha

El código fuente y la documentación deberán mantenerse en un único **repositorio público**. La versión final evaluable deberá encontrarse en la rama principal. El grupo deberá definir y utilizar una estrategia de ramas, revisiones e integración acorde con su forma de trabajo.

El sistema deberá poder ponerse en marcha localmente mediante un procedimiento único, automatizado y documentado. Ese procedimiento deberá crear o iniciar las dependencias necesarias y no podrá depender de configuraciones manuales que solo existan en las computadoras de los integrantes. Los secretos y datos sensibles no deberán almacenarse en el repositorio.

3. Documentación

La documentación forma parte del sistema y será evaluada junto con el código. Deberá mantenerse actualizada durante el desarrollo; no deberá construirse únicamente al finalizar el proyecto.

Archivo	Contenido Mínimo
README.md	Presentación general del proyecto: Introduce el dominio, el objetivo y el flujo principal. Explica cómo ejecutar el sistema localmente, cómo acceder a la parte desplegada y dónde encontrar el resto de la documentación.
SPEC.md o artefacto equivalente	Definición del alcance y los requisitos: Registra las funcionalidades, las reglas de negocio y los criterios de aceptación del sistema. Su formato y ubicación dependerán de la forma de trabajo elegida por el grupo: podrá resolverse mediante una especificación, historias de usuario, casos de uso, un backlog u otros artefactos equivalentes. No se exige un archivo `SPEC.md` cuando la metodología seleccionada ya produzca documentación que cumpla la misma finalidad.
docs/ARCHITECTURE.md	Explica la arquitectura general mediante texto y diagramas. Debe permitir reconocer los servicios, sus responsabilidades, los datos que administra cada uno, sus comunicaciones y la distribución de los componentes.
docs/adr/ADR-XXX.md	Registro de decisiones arquitectónicas: Documenta las decisiones obligatorias y cualquier otra decisión relevante mediante el formato definido en la sección 8. Cada decisión se conserva como un archivo independiente
docs/contracts/README.md	Documentación de la capacidad publicada y contrato versionado: Reúne el contrato utilizado para la integración, sus versiones, ejemplos de uso, posibles errores y la información necesaria para que otro grupo pueda consumir la capacidad.

docs/POSTMORTEM.md	Informe de la caída provocada: Analiza lo ocurrido durante la jornada de fallas controladas: impacto, línea temporal, detección, respuesta, causa y acciones de mejora.
--------------------	---

4. Integración entre grupos

4.1 Organización de la integración

Cada grupo deberá:

1. Publicar y documentar una capacidad de su sistema para que sea utilizada por otro grupo;
2. Consumir una capacidad publicada por un grupo diferente.

Los profesores asignarán las relaciones entre proveedores y consumidores.

Publicar una capacidad implica poner a disposición una forma definida de utilizarla, para permitir que el grupo consumidor se integre sin depender de explicaciones privadas. La primera versión operativa de la capacidad deberá encontrarse en un entorno accesible y mantenerse operativa hasta finalizar la evaluación.

La capacidad deberá contar con un contrato formal y versionado, almacenado junto con su documentación en `docs/contracts/`. Este contrato podrá describir una API, eventos o ambos y deberá utilizar un formato estándar y procesable adecuado para el mecanismo de comunicación elegido.

El contrato y su documentación deberán indicar, según corresponda:

- qué operaciones o eventos se encuentran disponibles;
- qué información debe enviarse y qué resultados pueden obtenerse;
- qué errores pueden producirse y cómo deben interpretarse;
- qué condiciones de autenticación, idempotencia o uso deben respetarse;
- qué versión se encuentra vigente;
- ejemplos suficientes para implementar y probar la integración.

La capacidad del proveedor deberá incorporarse en algún flujo importante del sistema consumidor. Una llamada aislada, una pantalla de prueba o una integración sin consecuencias sobre el negocio no serán consideradas válidas.

La comunicación con el proveedor deberá realizarse desde el microservicio responsable de la funcionalidad que utiliza esa capacidad. No deberá iniciarse directamente desde el frontend ni canalizarse a través del punto de entrada propio del sistema.

El consumidor deberá trabajar a partir del contrato y la documentación publicados.

El grupo consumidor deberá:

- implementar la integración respetando el contrato publicado;
- incluir un test de contrato capaz de detectar cambios incompatibles;
- definir tiempos de espera y tratamiento de errores;
- decidir qué observa el usuario cuando el proveedor falla;
- demostrar ese comportamiento durante la jornada de fallas controladas;

Esta integración reproduce una situación en la que un equipo debe trabajar con un sistema que no controla. A su vez, publicar una capacidad propia obligará al grupo a producir documentación utilizable por personas que no participaron de sus decisiones internas.

5. Registro y justificación de decisiones arquitectónicas

Los registros de decisiones arquitectónicas (ADR) tienen como finalidad explicar por qué se eligió una alternativa, qué otras opciones se evaluaron y qué consecuencias se aceptaron.

Las decisiones deberán registrarse durante el diseño y la implementación, y mantenerse actualizadas en el hito correspondiente. En principio, cada decisión obligatoria se documentará en un ADR independiente. El grupo podrá agregar otros registros cuando identifique decisiones relevantes que no se encuentren contempladas en la tabla.

#	Decisión arquitectónica	Aspectos que deberán quedar justificados
D1	Límites de los servicios	Criterios utilizados para separar responsabilidades, relaciones entre servicios y propiedad de los datos.
D2	Arquitectura interna de los servicios	Estilos o patrones seleccionados, dependencias internas y adecuación de cada alternativa al problema del servicio.
D3	Persistencia	Tipo de almacenamiento elegido para cada servicio, patrones de acceso que debe soportar y limitaciones aceptadas.
D4	Consistencia y concurrencia	Operaciones que requieren garantías especiales y tratamiento de concurrencia, duplicación, pérdida y fallas parciales.
D5	Comunicación entre servicios	Uso de interacciones síncronas y asíncronas, tiempos de espera, reintentos, eventos e idempotencia.

#	Decisión arquitectónica	Aspectos que deberán quedar justificados
D6	Búsqueda	Información indexada, criterios de consulta, actualización o reconstrucción del índice y retraso aceptable.
D7	Cache	Información almacenada temporalmente, invalidación, vigencia, comportamiento ante fallas y criterios de medición.
D8	Contrato propio	Diseño, publicación, compatibilidad y estrategia de versionado de la capacidad ofrecida a otro grupo.
D9	Consumo del proveedor	Incorporación de la capacidad externa, adaptación al contrato y comportamiento ante errores, cambios o indisponibilidad.
D10	Resiliencia	Comportamiento ante fallas y mecanismos seleccionados para limitar su propagación o degradar el sistema de manera controlada.
D11	Observabilidad	Estrategia de logs, métricas, trazas y tableros, junto con el objetivo de servicio y los criterios de alerta.
D12	Balanceo de carga	Servicio seleccionado, estrategia de distribución, verificación de disponibilidad y comportamiento ante la caída de una instancia.
D13	Capacidad y costos	Resultados de las pruebas de carga, principal límite identificado, alternativa de escalado y estimación de costos.

Si una decisión cambia, no deberá borrarse su historia. El ADR anterior se marcará como reemplazado y el nuevo registro deberá indicar qué decisión reemplaza y por qué fue necesario revisarla.

6. Cronograma y Entregables

6.1 Viernes 2 de octubre - Instancia Inicial

El grupo deberá contar con:

- integrantes confirmados;
- dominio y alcance preliminar aprobados;
- repositorio creado y accesible para la cátedra;

6.2 Viernes 9 de octubre - Entrega 1: Diseño y contrato con mock

El grupo deberá presentar:

- primera versión del `README.md`, de la documentación de alcance correspondiente a la forma de trabajo elegida y de la descripción de arquitectura (`docs/[ARCHITECTURE.md](#)`);
- diagramas de contexto y de contenedores;
- límites preliminares de los servicios, sus responsabilidades y la propiedad de los datos;
- capacidad propia a exponer seleccionada y documentada;
- estructura inicial de los servicios y sus dependencias;
- decisiones D1 y D8, junto con versiones iniciales de D3 y D5;

6.3 Viernes 23 de octubre - Entrega 2: Dominio propio de punta a punta

El grupo deberá presentar:

- al menos un servicio operativos, además la funcionalidad a compartir funcionando;
- al menos un tipo de almacenamiento integrados al flujo;
- logs estructurados y correlacionados, junto con una primera traza distribuida del flujo principal;
- decisiones D2, D6, D7, D9, D10, D12 y D13; validación de D1, D3 y D5; primera versión de D11 y actualización de D8 si el contrato hubiera cambiado.

Miércoles 11 o viernes 13 de noviembre: Presentacion Grupal

La fecha correspondiente a cada grupo se sorteará y comunicará el 6 de noviembre.

Incluirá:

- frontend completo, desde el ingreso del usuario hasta la funcionalidad compartida;
- punto de entrada único con autenticación, direccionamiento y control de tráfico;
- al menos tres servicios operativos;
- implementación real de la capacidad propia, desplegada en la nube y accesible mediante una URL pública;
- capacidad del proveedor incorporada al flujo principal,
- al menos dos tipos de almacenamiento integrados al flujo;

- caché integrada en un flujo de lectura relevante, con medición comparativa antes y después;
- dos estilos o patrones de arquitectura interna aplicados de manera reconocible;
- evento de dominio publicado y consumido mediante mensajería, con consumidor idempotente y tratamiento de mensajes no procesables;
- búsqueda indexada con paginación, filtros, ordenamiento y mecanismo de actualización o reconstrucción del índice;
- verificación de las decisiones mediante la evidencia obtenida sobre el sistema;
- sección de limitaciones conocidas y deuda técnica aceptada, actualizada en la descripción de la arquitectura (`docs/ARCHITECTURE.md`).

Defensa Individual (Examen Final)

- operación crítica con las garantías de consistencia y concurrencia definidas;
- tests unitarios, un test de integración contra una dependencia real;
- test de contrato sobre la capacidad consumida;
- balanceo de carga operativo sobre dos o más instancias de al menos un servicio, con verificación de disponibilidad y evidencia de la distribución de solicitudes;
- simulación de la caída de una dependencia propia, del proveedor externo o de ambas;
- test de carga ejecutado, con resultados, análisis de capacidad, principal límite identificado;
- comportamiento documentado ante la caída de cada dependencia relevante y al menos un mecanismo de protección implementado;
- logs, métricas y trazas completos, junto con el tablero y la alerta que se utilizarán durante la presentación;
- evidencia de una caída controlada realizada como ensayo y su informe (`docs/[POSTMORTEM.md](#)`);

7. Bonus

Cada grupo podrá implementar hasta dos desafíos adicionales, con un valor máximo de hasta un punto por desafío. Deberán anunciarse en la Entrega 2 e incorporarse al alcance documentado del proyecto.

Algunas alternativas posibles son:

-
- un agente que ejecute operaciones del sistema mediante un protocolo estándar;
 - actualizaciones de información en tiempo real;
 - una capa de agregación adaptada a las necesidades de la interfaz;
 - orquestación de contenedores;
 - creación de tests e2e;
 - escalado automático;

Para otorgar puntaje, el desafío deberá estar integrado al sistema, documentado, testeado y defendido.